

Contents

Doc-tools	1
Usage	1
Contributing	1
1 - Statistics	2
2 - Documentation	2
Usage	3
HTML	5
PDF	5
3 - Formatting	6

Doc-tools

This repository contains useful tooling for/when developing a (mostly) C/C++ project.

Currently includes:

1. Statistics
 1. **tokei** - line of code counting.
2. Documentation
 1. **doxygen** - documentation generation from the project's source code.
3. Formatting
 1. **clang-format** - formats the code accordingly to a configuration file.

Usage

- Alternatively see the branch **workflow-example** for an usage example (in portuguese).

Download and extract as a directory named **doc-tools** inside of your C/C++ project, at root level.

If you clone it, you'll have the **doc-tools/.git** directory as well but you may delete it.

Note that the **doc-tools/.gitignore** filters everything from within **doc-tools/**.

This way, when **doc-tools** is added into some project **X**, the **doc-tools'** source won't have to be part of **X's** source code.

Contributing

After changing/adding/removing the desired files, you may either *force* (**-f**) add it, or change the **doc-tools/.gitignore** file in order to disable the "ignore ev-

everything" (**) filter, so that your last changes may be added and then committed. In the latter case, after committing your changes, please re-enable the "ignore everything" filter in a new, posterior commit.

1 - Statistics

To show line count statistics, run `tokei.bat`.

Requires:

- Nothing.

It is standalone and wraps the `tokei.exe` file, which was compiled from it's repository¹.

2 - Documentation

The documentation may generate `html` or `pdf` files, please see their respective sub-sections for more information.

Requires:

- `doxygen` command. Download page².
- `graphviz` package, for the DOT rendering. Download page³.
 - You still need to change your PATH manually: Check the file path at `C:\Program Files (x86)\Graphvizx.xx\bin\dot.exe` and copy that directory path (which ends in `\bin`), then open Control Panel > System and Security > System > Advanced system settings > Advanced > Environment Variables, and then in User variables for <user> select Variable Path and Edit it, then add a New path and then paste the copied directory path. Then Ok a few times and that's it.
 - If you cannot install this software, you may change the `Doxyfile.txt` and disable the `HAS_DOT` setting, and then `graphviz` will not be required by documentation generation.
- (optional) `pdflatex` command, included on MiKTeX. Download page⁴.
 - This is only required if you want to be able to generate the PDF output.
 - You still need to install custom latex-related "packages". When first trying to generate the PDF outputs, MiKTeX will automatically prompt you to install third-party packages. During that prompt, you may want to choose a HTTPS repository (such as

¹<https://github.com/XAMPPRocky/tokei>

²<https://sourceforge.net/projects/doxygen/files/latest/download>

³https://graphviz.gitlab.io/_pages/Download/Download_windows.html

⁴<https://miktex.org/download>

`https://ctan.math.illinois.edu`) and setup proxy settings (see **Start > Proxy Settings > Manual proxy setup** **Address** and **Port** fields for proxy information, eg. `http://lnx237in.sjk.emb` and 9090), and also check “authentication” for the proxy (and then inserting your login and password when prompted).

- After all required packages are installed (there are about 30 packages), latex will continue the rendering procedure. Any error in this situation is specific to latex parsing/rendering.

- * One possible error, as of 2019, is related to this issue⁵. A workaround, as suggested, is to download a patched `tabu.sty`⁶ and replace it into your local latex package (the `tabu` package, in this case). First, you need to find your “old” `tabu.sty` file. You may find it by opening **MiKTeX Console > Settings > Directories > TEXMF root directories**, and select a line which the **Purposes** column indicates **Install**; then Right-click it and select **Open**. If this does not work, you may try other lines with the same **Install Purposes**. In my case, this is `C:\Users\<user>\AppData\Local\Programs\MiKTeX 2.9\`. Then enter the directories `tex/latex/tabu` and rename `tabu.sty` to `tabu.bak.sty` (for a backup), and then paste the downloaded fixed/patched `tabu.sty`.

- * I have encountered no other errors during PDF generation. If you have encountered and also found a solution, please try taking a moment to update this document.

- `config/` directory for the `doxygen` settings.
- `Doxyfile.custom.txt`, which is the preferred configuration file for adding your own customization.
 - The project title and also the file path patterns to exclude may be defined here.

It utilizes `doxygen` to generate the html (or pdf) documentation accordingly to the configuration files from `config/` directory.

The generated documentation is contained at `rendered/html/` (or `rendered/private/html/`) directory.

It then opens the `rendered/html/index.html` (or `rendered/private/html/index.html`) file with the browser.

It behaves similarly to the PDF case.

Usage

Refer to the `doxygen` manual⁷ on how to add documentation commentaries. Both Javadoc-Style and Markdown are enabled per the default configuration `config/Doxyfile.txt`. It is also configured to consider the first paragraph to

⁵<https://github.com/doxygen/doxygen/issues/6769>

⁶<https://raw.githubusercontent.com/tabu-fixed/tabu/master/tabu.sty>

⁷<http://www.doxygen.nl/manual/docblocks.html>

be used as a briefed description of that documented item.

Markdown Example using Markdown-style⁸:

```
// This line is a normal commentary which is not included in the documentation

/// This is a brief description of the namespace below.
/// Only after a blank line the brief description ends.
///
/// After a blank line, other doc-commentaries are "the expanded" documentation.
///
/// Simply adding a new line still outputs a continuous documentation.
/// For an actual new line,
/// you need to put two extra spaces at the line of the line.
///
/// If you need to refer to other items, such as function `y` below,
/// use the backticks so that it will be rendered in monospace style.
namespace x {}

/// This is a brief description of the function below.
void y(
    /// the first argument description.
    char a,
    /// the second argument description.
    char b,
) {};
```

Markdown files On your project, try making sure that there is only one README.md, which will be rendered as the project main page. Other markdown files, with their own names, will each be rendered as a “related page” in the documentation.

In such “related pages” markdowns, as expressed in the doxygen markdown documentation⁹, you may prefer to add label after the heading.

For example, consider you have a namespace `z` and you also have a document page for that namespace.

You may have a file named `z.md` and `z.h`.

```
z.md

# Z {#z_md}
\brief Documentation for @ref z.
```

Z's description, references, etc, in standard markdown.
text text text text text text text...

⁸<http://www.doxygen.nl/manual/markdown.html>

⁹http://www.doxygen.nl/manual/markdown.html#md_header_id

z.h

```
/// See @ref z_md for more information.  
namespace z {}
```

This way, the namespace **z** will contain a link to the (related page) documentation generated by **z.md**,
and the (related page) documentation generated by **z.md** will contain a link to the namespace **z**.

Specific pages are viable if you don't want to insert a lot of documentation (let's say, 100+ lines of documentation) directly into the source code.

Javadoc Example using Javadoc-style:

```
// This line is a normal commentary which is not included in the documentation  
  
/**  
 * This is a brief description of the namespace below.  
 * Only after a blank line the brief description ends.  
 *  
 * After a blank line, other doc-commentaries are "the expanded" documentation.  
 */  
namespace x {}  
  
/**  
 * This is a brief description of the function below.  
 * @param a the first argument description.  
 * @param b the second argument description.  
 */  
void y(char a, char b) {};
```

HTML

To generate and open the **html** documentation, run **generate_then_open.html.bat**.

For also including private members in the documentation, run **generate_then_open.private.html.bat**.

Note that these are considered two different and independent documentations.

PDF

The PDF documentation requires the **pdflatex** command, of which you can read about its installation and configuration at the start of this section (2 - Documentation).

To generate and open the **pdf** documentation, run **generate_then_open.pdf.bat**.

For also including private members in the documentation, run **generate_then_open.private.pdf.bat**.

Note that these are considered two different and independent documentations.

3 - Formatting

To format all of the source files, recursively, run `clang-format.replace.sh` linux command.

Requires:

- LLVM framework. Windows installer (64-bit)¹⁰.
- `Git Bash` (for windows).
- `clang-format`, which comes bundled with LLVM.

This is a linux-based shell command which finds and applies a formatting command to the parent directory source files, recursively.

`Git Bash` is required because it is easier to find and apply commands to files in a recursive way;

The actual formatting command is `clang-format`.

Finally, the `clang-format` file is used as a formatting configuration.

¹⁰<https://llvm.org/builds/>